

Course Name: Principles of Software Design

Course Code: ENSF 480

Assignment Number: Lab 1

Lab Section: B01

Completed by:

Aayush Karthikeyan (UCID# 30189743)

Sarvesh Vettrivelan (UCID# 30242015)

Submission Date: 14/09/2026

Submission Time:

Exercise B: Dictionary Linked List Implementation

1. Source Code

a. dictionaryList.h

```
// File Name: dictionaryList.h
// Assignment: Lab 1 Exercise B
// Lab Section: LAB B01
// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID# 30242015)
// Submission Date: 14/09/2026 23:59

#ifndef DICTIONARYLIST_H
#define DICTIONARYLIST_H

#include <string>

typedef int Key;
typedef std::string Datum;

class Node {
    friend class DictionaryList;

private:
    Key keyM;
    Datum datumM;
    Node *nextM;

    Node(const Key& keyA, const Datum& datumA, Node *nextA);
};

// A sorted list of unique keys and their associated data.
class DictionaryList {
public:
    DictionaryList();
    DictionaryList(const DictionaryList& source);
    DictionaryList& operator=(const DictionaryList& rhs);
    ~DictionaryList();

    int size() const;
    int cursor_ok() const;
    // These two accessors require a valid cursor.
```

```

const Key& cursor_key() const;
Datum& cursor_datum() const;

// Inserting or removing a key invalidates the cursor.
void insert(const Key& keyA, const Datum& datumA);
void remove(const Key& keyA);
void go_to_first();
void step_fwd(); // Requires a valid cursor.
void make_empty();
void find(const Key& keyA); // Invalidates the cursor if the key is absent.

private:
    int sizeM;
    Node *headM;
    Node *cursorM;

    void destroy();
    // Copies nodes and cursor position into an empty list.
    void copy(const DictionaryList& source);
};

#endif

```

b. dictionaryList.cpp

```

// File Name: dictionaryList.cpp
// Assignment: Lab 1 Exercise B
// Lab Section: LAB B01
// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID# 30242015)
// Submission Date: Sept 14, 2026

```

```

#include "dictionaryList.h"
#include <cassert>

```

```

using namespace std;

```

```

Node::Node(const Key& keyA, const Datum& datumA, Node *nextA)
    : keyM(keyA), datumM(datumA), nextM(nextA)
{
}

```

```

DictionaryList::DictionaryList()
    : sizeM(0), headM(nullptr), cursorM(nullptr)
{
}

```

```
}
```

```
DictionaryList::DictionaryList(const DictionaryList& source)
: sizeM(0), headM(nullptr), cursorM(nullptr)
{
    copy(source);
}
```

```
DictionaryList& DictionaryList::operator =(const DictionaryList& rhs)
{
    if (this != &rhs) {
        destroy();
        copy(rhs);
    }
    return *this;
}
```

```
DictionaryList::~~DictionaryList()
{
    destroy();
}
```

```
int DictionaryList::size() const
{
    return sizeM;
}
```

```
int DictionaryList::cursor_ok() const
{
    return cursorM != nullptr;
}
```

```
const Key& DictionaryList::cursor_key() const
{
    assert(cursor_ok());
    return cursorM->keyM;
}
```

```
Datum& DictionaryList::cursor_datum() const
{
    assert(cursor_ok());
    return cursorM->datumM;
}
```

```
void DictionaryList::insert(const int& keyA, const string& datumA)
```

```
{  
    // Add new node at head?  
    if (headM == nullptr || keyA < headM->keyM) {  
        headM = new Node(keyA, datumA, headM);  
        sizeM++;  
    }  
    // Overwrite datum at head?  
    else if (keyA == headM->keyM) {  
        headM->datumM = datumA;  
    }  
    // Search downstream  
    else {  
        for (Node *p = headM; p != nullptr; p = p->nextM) {  
            if (keyA == p->keyM) {  
                p->datumM = datumA;  
                cursorM = nullptr;  
                return;  
            }  
        }  
    }
```

```
    Node *p = headM->nextM;
```

```
    Node *prev = headM;
```

```
    while (p != nullptr && keyA > p->keyM) {  
        prev = p;  
        p = p->nextM;  
    }
```

```
    prev->nextM = new Node(keyA, datumA, p);  
    sizeM++;
```

```
    }  
    cursorM = nullptr;  
}
```

```
void DictionaryList::remove(const int& keyA)
```

```
{  
    cursorM = nullptr;  
  
    if (headM == nullptr || keyA < headM->keyM)  
        return;
```

```
    Node *doomed_node = nullptr;
```

```

if (keyA == headM->keyM) {
    doomed_node = headM;
    headM = headM->nextM;
}
else {
    Node *before = headM;
    Node *maybe_doomed = headM->nextM;
    while (maybe_doomed != nullptr && keyA > maybe_doomed->keyM) {
        before = maybe_doomed;
        maybe_doomed = maybe_doomed->nextM;
    }

    if (maybe_doomed != nullptr && maybe_doomed->keyM == keyA) {
        doomed_node = maybe_doomed;
        before->nextM = maybe_doomed->nextM;
    }
}

if (doomed_node != nullptr) {
    delete doomed_node;
    sizeM--;
}
}

void DictionaryList::go_to_first()
{
    cursorM = headM;
}

void DictionaryList::step_fwd()
{
    assert(cursor_ok());
    cursorM = cursorM->nextM;
}

void DictionaryList::make_empty()
{
    destroy();
}

void DictionaryList::find(const Key& keyA)
{
    for (Node *p = headM; p != nullptr; p = p->nextM) {
        if (keyA == p->keyM) {

```

```

        cursorM = p;
        return;
    }
}
cursorM = nullptr;
}

```

```

void DictionaryList::destroy()
{
    Node *p = headM;
    Node *prev = nullptr;
    while (p != nullptr) {
        prev = p;
        p = p->nextM;
        delete prev;
    }
    headM = nullptr;
    cursorM = nullptr;
    sizeM = 0;
}

```

```

void DictionaryList::copy(const DictionaryList& source)
{
    headM = nullptr;
    cursorM = nullptr;
    sizeM = 0;

```

```

    if (source.headM == nullptr)
        return;

```

```

    try {
        headM = new Node(source.headM->keyM, source.headM->datumM, nullptr);
        Node *newest = headM;
        sizeM++;

```

```

    if (source.headM == source.cursorM)
        cursorM = newest;

```

```

    for (const Node *src = source.headM->nextM; src != nullptr; src = src->nextM) {
        newest->nextM = new Node(src->keyM, src->datumM, nullptr);
        newest = newest->nextM;
        sizeM++;

```

```

    if (src == source.cursorM)

```

```

        cursorM = newest;
    }
}
catch (...) {
    // A failed constructor will not call this object's destructor.
    destroy();
    throw;
}
}
}

```

c. exBmain.cpp

```

// File Name: exBmain.cpp
// Assignment: Lab 1 Exercise B
// Lab Section: LAB B01
// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID#
30242015)
// Submission Date: 14/09/2026 23:59

```

```

#include <assert.h>
#include <iostream>
#include "dictionaryList.h"
using namespace std;
DictionaryList dictionary_tests();
void test_copying();
void print(DictionaryList& dl);
void test_finding(DictionaryList& dl);

int main()
{
    DictionaryList dl = dictionary_tests();

    #if 1
        test_copying();
    #endif

    #if 1
        test_finding(dl);
    #endif
    return 0;
}

DictionaryList dictionary_tests()
{
    DictionaryList dl;

```

```

assert(dl.size() == 0);
cout << "\nPrinting list just after its creation ...\n";
print(dl);

dl.insert(8001,"Dilbert");
dl.insert(8002,"Alice");
dl.insert(8003,"Wally");
assert(dl.size() == 3);
cout << "\nPrinting list after inserting 3 new keys ...\n";
print(dl);
dl.remove(8002);
dl.remove(8001);
dl.insert(8004,"PointyHair");
assert(dl.size() == 2);
cout << "\nPrinting list after removing two keys and inserting PointyHair ...\n";
print(dl);

dl.insert(8003,"Sam");
assert(dl.size() == 2);
cout << "\nPrinting list after changing data for one of the keys ...\n";
print(dl);

dl.insert(8001,"Allen");
dl.insert(8002,"Peter");
assert(dl.size() == 4);
cout << "\nPrinting list after inserting 2 more keys ...\n";
print(dl);

cout << "***----Finished dictionary tests-----***\n\n";
return dl;
}

void test_copying()
{
    DictionaryList one;
    DictionaryList two;
    assert(two.size() == 0);

    one.insert(319,"Randomness");
    one.insert(315,"Shocks");
    one.insert(335,"ParseErrors");
    one.go_to_first();
    one.step_fwd();

```

```

DictionaryList three(one);
assert(three.cursor_datum() == "Randomness");
one.remove(335);

cout << "Printing list--keys should be 315, 319\n";
print(one);

cout << "Printing list--keys should be 315, 319, 335\n";
print(three);

one = two = three = three;
one.remove(319);
two.remove(315);

cout << "Printing list--keys should be 315, 335\n";
print(one);

cout << "Printing list--keys should be 319, 335\n";
print(two);

cout << "Printing list--keys should be 315, 319, 335\n";
print(three);

cout << "***----Finished tests of copying-----***\n\n";
}

void print(DictionaryList& dl)
{
    if (dl.size() == 0)
        cout << " List is EMPTY.\n";
    for (dl.go_to_first(); dl.cursor_ok(); dl.step_fwd()) {
        cout << " " << dl.cursor_key();
        cout << " " << dl.cursor_datum().c_str() << '\n';
    }
}

void test_finding(DictionaryList& dl)
{
    cout << "\nLet's look up some names ...\n";

    dl.find(8001);
    if (dl.cursor_ok())
        cout << " name for 8001 is: " << dl.cursor_datum().c_str() << ".\n";
    else

```

```

        cout << " Sorry, I couldn't find 8001 in the list. \n";

    dl.find(8000);
    if (dl.cursor_ok())
        cout << " name for 8000 is: " << dl.cursor_datum().c_str() << ".\n";
    else
        cout << " Sorry, I couldn't find 8000 in the list. \n";

    dl.find(8002);
    if (dl.cursor_ok())
        cout << " name for 8002 is: " << dl.cursor_datum().c_str() << ".\n";
    else
        cout << " Sorry, I couldn't find 8002 in the list. \n";

    dl.find(8004);
    if (dl.cursor_ok())
        cout << " name for 8004 is: " << dl.cursor_datum().c_str() << ".\n";
    else
        cout << " Sorry, I couldn't find 8004 in the list. \n";

    cout << "***----Finished tests of finding -----***\n\n";
}

```

2. Program Execution Output

Printing list just after its creation ...

List is EMPTY.

Printing list after inserting 3 new keys ...

8001 Dilbert

8002 Alice

8003 Wally

Printing list after removing two keys and inserting PointyHair ...

8003 Wally

8004 PointyHair

Printing list after changing data for one of the keys ...

8003 Sam

8004 PointyHair

Printing list after inserting 2 more keys ...

8001 Allen

8002 Peter

8003 Sam

8004 PointyHair

-----Finished dictionary tests-----

Printing list--keys should be 315, 319

315 Shocks

319 Randomness

Printing list--keys should be 315, 319, 335

315 Shocks

319 Randomness

335 ParseErrors

Printing list--keys should be 315, 335

315 Shocks

335 ParseErrors

Printing list--keys should be 319, 335

319 Randomness

335 ParseErrors

Printing list--keys should be 315, 319, 335

315 Shocks

319 Randomness

335 ParseErrors

----Finished tests of copying-----

Let's look up some names ...

name for 8001 is: Allen.

Sorry, I couldn't find 8000 in the list.

name for 8002 is: Peter.

name for 8004 is: PointyHair.

----Finished tests of finding -----

Exercise C: Refactoring Using Four OOP Pillars

Source Code (company.cpp)

// File Name: company.cpp

// Assignment: Lab 1 Exercise C

// Lab Section: LAB B01

// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID# 30242015)

// Submission Date: 14/09/2026 23:59

#include <string>

#include <vector>

using namespace std;

// Class encapsulating calendar date representation

class Date {

private:

int day;

int month;

int year;

};

// Class encapsulating physical address components

class Address {

private:

```
    string street;  
    string city;  
    string province;  
    string postalCode;  
};
```

// Base class modeling shared human attributes

```
class Person {  
private:  
    string name;  
    Address address;  
};
```

// Derived class demonstrating Hierarchy: Employee IS-A Person

```
class Employee : public Person {  
public:  
    enum class State {  
        ACTIVE,  
        SUSPENDED,  
        RETIRED,  
        FIRED  
    };  
};
```

```
private:
```

```
    Date dateOfBirth;  
  
    State state;  
  
};
```

```
// Derived class demonstrating Hierarchy: Customer IS-A Person
```

```
class Customer : public Person {  
  
private:  
  
    string phone;  
  
};
```

```
// Class representing Company aggregation and composition
```

```
class Company {  
  
private:  
  
    string companyName;  
  
    Address companyAddress;  
  
    Date dateEstablished;  
  
    vector<Employee> employees;  
  
    vector<Customer> customers;  
  
};
```

Exercise D: Class Refactoring, Separation, and Memory Management

1. Source Code

a. point.h

// File Name: point.h

// Assignment: Lab 1 Exercise D

// Lab Section: LAB B01

// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID# 30242015)

// Submission Date: Sept 14, 2026

#ifndef POINT_H

#define POINT_H

class Point {

private:

double x; // x coordinate of a location on Cartesian Plane

double y; // y coordinate of a location on Cartesian Plane

public:

Point(double a = 0.0, double b = 0.0);

double get_x() const;

double get_y() const;

void set_x(double a);

void set_y(double a);

};

```
#endif
```

b. point.cpp

```
// File Name: point.cpp
```

```
// Assignment: Lab 1 Exercise D
```

```
// Lab Section: LAB B01
```

```
// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID# 30242015)
```

```
// Submission Date: Sept 14, 2026
```

```
#include "point.h"
```

```
Point::Point(double a, double b) : x(a), y(b) {}
```

```
double Point::get_x() const {  
    return x;  
}
```

```
double Point::get_y() const {  
    return y;  
}
```

```
void Point::set_x(double a) {  
    x = a;
```

```
}
```

```
void Point::set_y(double a) {
```

```
    y = a;
```

```
}
```

c. human.h

```
// File Name: human.h
```

```
// Assignment: Lab 1 Exercise D
```

```
// Lab Section: LAB B01
```

```
// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID# 30242015)
```

```
// Submission Date: Sept 14, 2026
```

```
#ifndef HUMAN_H
```

```
#define HUMAN_H
```

```
#include "point.h"
```

```
class Human {
```

```
private:
```

```
    Point location; // Location of an object of Human on a Cartesian Plane
```

```
    char *name;     // Owned, null-terminated name; released by the destructor.
```

```
public:
```

```
Human(const char* nam = "", double x = 0.0, double y = 0.0);

~Human();

Human(const Human& src);

Human& operator=(const Human& rhs);


// Returned text is valid until the name changes or this Human is destroyed.

const char* get_name() const;

void set_name(const char* nam);

Point get_point() const;

void display() const;

};

#endif
```

d. human.cpp

```
// File Name: human.cpp

// Assignment: Lab 1 Exercise D

// Lab Section: LAB B01

// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID#
30242015)

// Submission Date: Sept 14, 2026


#include "human.h"

#include <cstring>
```

```
#include <iostream>
```

```
using namespace std;
```

```
Human::Human(const char* nam, double x, double y)
```

```
    : location(x, y), name(nullptr)
```

```
{
```

```
    set_name(nam == nullptr ? "" : nam);
```

```
}
```

```
Human::~~Human() {
```

```
    delete[] name;
```

```
}
```

```
Human::Human(const Human& src)
```

```
    : location(src.location), name(new char[strlen(src.name) + 1])
```

```
{
```

```
    strcpy(this->name, src.name);
```

```
}
```

```
Human& Human::operator=(const Human& rhs) {
```

```
    if (this != &rhs) {
```

```
        set_name(rhs.name);
```

```
        location = rhs.location;
```

```

    }

    return *this;
}

const char* Human::get_name() const {
    return name;
}

void Human::set_name(const char* nam) {
    if (nam == nullptr) return;

    // Copy first: nam may point into the current name buffer.
    char *new_name = new char[strlen(nam) + 1];

    strcpy(new_name, nam);

    delete[] name;

    name = new_name;
}

Point Human::get_point() const {
    return location;
}

void Human::display() const {
    cout << "Human Name: " << name << "\nHuman Location: "
        << location.get_x() << " , "

```

```
        << location.get_y() << ".\n" << endl;
    }
}
```

e. main.cpp

// File Name: main.cpp

// Assignment: Lab 1 Exercise D

// Lab Section: LAB B01

// Completed by: Aayush Karthikeyan (UCID# 30189743), Sarvesh Vettrivelan (UCID# 30242015)

// Submission Date: Sept 14, 2026

```
#include "human.h"
```

```
int main() {
    double x = 2000, y = 3000;
    Human h("Ken Lai", x, y);
    h.display();
    return 0;
}
```

2. Program Execution Output

Human Name: Ken Lai

Human Location: 2000 ,3000.